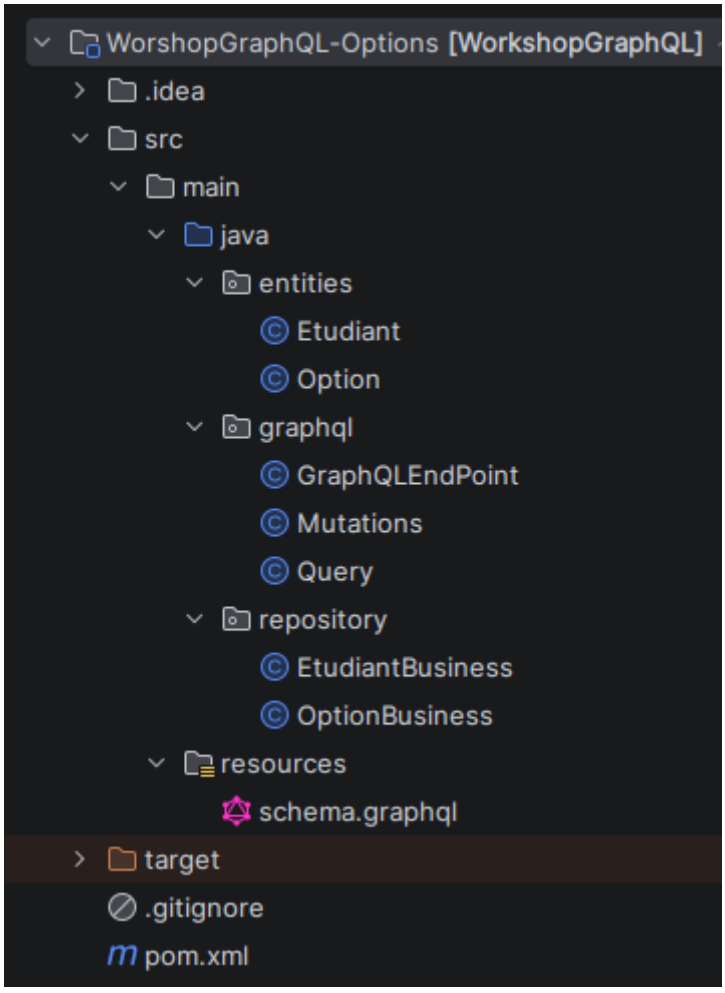
esprit Se former autrement HONORIS UNITED UNIVERSITIES	Année Universitaire : 2026-2027 **Architecture Orientée Services**

Configuration

Génération d'une application GraphQL

La mise en œuvre de l'application

Le projet doit avoir la structure **finale** suivante :



1. Ajouter les dépendances suivantes au fichier POM :

```
<dependencies>
<dependency>
<groupId>com.graphql-java</groupId>
<artifactId>graphql-java</artifactId>
<version>3.0.0</version>
</dependency>
<dependency>
<groupId>com.graphql-java</groupId>
<artifactId>graphql-java-tools</artifactId>
<version>3.2.0</version>
</dependency>
<dependency>
<groupId>com.graphql-java</groupId>
<artifactId>graphql-java-servlet</artifactId>
<version>4.0.0</version>
</dependency>
<dependency>
<groupId>javax.servlet</groupId>
<artifactId>javax.servlet-api</artifactId>
<version>3.0.1</version>
<scope>provided</scope> </dependency> </dependencies>
```

2. Création de la classe Query :

Créer la classe « Query.java » dans le package « **graphql** »

3. Création du fichier schema.graphqls :

Créer le fichier « schema.graphqls » dans **src/main/ressources** :

```
type Option {
    codeOption: Int!
```

```
    libelle: String!
    domaine: String!
    responsable: String!
    credits: Int!
    semestre: Int!
    capacite: Int!
}

type Etudiant {
    identifiant: String!
    nom: String!
    prenom: String!
    anneeEtude: Int!
    email: String
    option: Option
}

type Query {
    getAllOptions: [Option!]!
}


schema {
    query: Query
}
```

4. Création de la classe GraphQLEndpoint :

Créer la classe « GraphQLEndpoint.java » dans le package « **graphql** ».

```java
package graphql;
import com.coxautodev.graphql.tools.SchemaParser;
import graphql.schema.GraphQLSchema;
import graphql.servlet.SimpleGraphQLServlet;
import repository.OptionBusiness;

import javax.servlet.annotation.WebServlet;

@WebServlet(urlPatterns = "/graphql")
public class GraphQLEndPoint extends SimpleGraphQLServlet {

    public GraphQLEndPoint() {
        super(buildSchema());
    }

    private static GraphQLSchema buildSchema() {

        OptionBusiness op= new OptionBusiness();

        return SchemaParser.newParser()
                .file("schema.graphql")
                .resolvers(new Query(op))
                .build()
                .makeExecutableSchema();

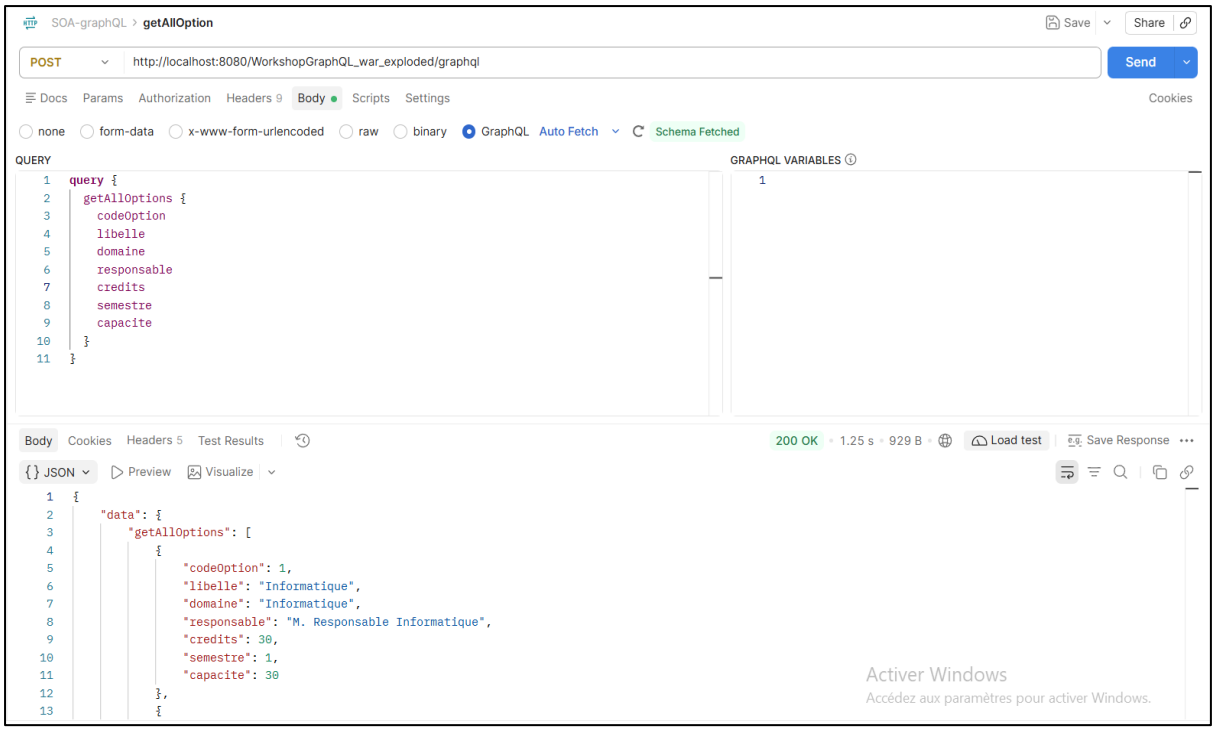
    }
}
```

NB :

@WebServlet est utilisée pour définir un composant Servlet dans une application Web. Cette annotation est spécifiée sur une classe et contient des métadonnées sur le servlet déclaré.

urlPatterns est le paramètre permettant de définir le chemin d'accès à la ressource.

5. **Déploiement :**
   Déployer le projet et tester l'url via **Postman** :
   http://localhost:port/nomProjet/graphql


SOA-graphQL > getAllOption
Save
Share
POST
http://localhost:8080/WorkshopGraphQL_war_exploded/graphql
Send
Docs Params Authorization Headers 9 Body Scripts Settings Cookies
none form-data x-www-form-urlencoded raw binary GraphQL Auto Fetch Schema Fetched
QUERY
GRAPHQL VARIABLES
query {
getAllOptions {
codeOption
libelle
domaine
responsable
credits
semestre
capacite
}
}
Body Cookies Headers 5 Test Results
200 OK 1.25 s 929 B
Load test
Save Response
JSON Preview Visualize
{
"data": {
"getAllOptions": [
{
"codeOption": 1,
"libelle": "Informatique",
"domaine": "Informatique",
"responsable": "M. Responsable Informatique",
"credits": 30,
"semestre": 1,
"capacite": 30
},
{
Activer Windows
Accédez aux paramètres pour activer Windows.